

Persuasion Dialogue System for Social Good

Objective

To build a dialogue system that can generate persuasive responses encouraging users to act for social good — e.g., charity donation, environment awareness — using the PersuasionForGood Corpus and a language model (GPT-Neo) fine-tuned with reinforcement learning (RLHF).

```
import pandas as pd
df=pd.read_csv('/content/full_dialog.csv')
```

```
import streamlit as st
from transformers import pipeline

# Load model
@st.cache_resource
def load_model():
    return pipeline("text-generation", model="./neo_persuasion_finetuned")

generator = load_model()

st.title("Persuasive Text Generator")

prompt = st.text_input("Enter user statement:")

if prompt:
    response = generator(prompt, max_length=80, do_sample=True, temperature=0.8)
    st.write("**AI Response:**")
    st.write(response[0]['generated_text'])
```

```
df.head()
# B2: Dialogue ID
# B4: Role (0 means persuader, 1 means persuadee)
# Turn: Turn index
# Unit: Sentence in utterance
```

	Unnamed: 0		Unit	Turn	B4	B2
0	0	Good morning. How are you doing today?	0	0		20180904-045349_715_live
1	1	Hi. I am doing good. How about you?	0	1		20180904-045349_715_live
2	2	I'm doing pretty good for a Tuesday morning.	1	0		20180904-045349_715_live
		Hehe. Same here, but it really feels				20180904

```

df = df.rename(columns={
    'Unit': 'text',
    'Turn': 'turn_id',
    'B4': 'role',
    'B2': 'dialogue_id'
})

# Drop unnecessary column
df = df.drop(columns=['Unnamed: 0'])

# Map roles for readability
df['role'] = df['role'].map({0: 'persuader', 1: 'persuadee'})

print(df.head())

```

	text	turn_id	role
0	Good morning. How are you doing today?	0	persuader
1	Hi. I am doing good. How about you?	0	persuadee
2	I'm doing pretty good for a Tuesday morning.	1	persuader
3	Haha. Same here, but it really feels like a Mo...	1	persuadee
4	Ugh yes it does!	2	persuader

	dialogue_id
0	20180904-045349_715_live
1	20180904-045349_715_live
2	20180904-045349_715_live
3	20180904-045349_715_live
4	20180904-045349_715_live

```

# Create input-output pairs
pairs = []
for i in range(len(df)-1):
    if df.iloc[i]['role'] == 'persuadee' and df.iloc[i+1]['role'] == 'persuader':
        pairs.append({
            'input_text': f"User: {df.iloc[i]['text']}",
            'target_text': df.iloc[i+1]['text']
        })

dataset = pd.DataFrame(pairs)
dataset.to_csv("formatted_pairs.csv", index=False)

```

```

from transformers import AutoTokenizer, AutoModelForCausalLM, Trainer, TrainingArguments
from datasets import Dataset

tokenizer = AutoTokenizer.from_pretrained("EleutherAI/gpt-neo-125M")
model = AutoModelForCausalLM.from_pretrained("EleutherAI/gpt-neo-125M")

hf_dataset = Dataset.from_pandas(dataset)

```

```

def preprocess(examples):
    # Concatenate input and target texts for each example in the batch

```

```

        texts = [examples["input_text"][i] + " " + examples["target_text"][i] for i in range(len(examples))]
        return tokenizer(texts, truncation=True, padding="max_length", max_length=MAX_LENGTH)

tokenizer.pad_token = tokenizer.eos_token
tokenized = hf_dataset.map(preprocess, batched=True)
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False)

args = TrainingArguments(
    output_dir="./neo_persuasion",
    per_device_train_batch_size=2,
    num_train_epochs=3,
    learning_rate=5e-5,
    logging_dir="./logs",
    save_strategy="epoch"
)

```

Map: 100%

10331/10331 [00:05<00:00, 1375.81 examples/s]

```

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized,
    tokenizer=tokenizer,
    data_collator=data_collator
)

trainer.train()
model.save_pretrained("./neo_persuasion_finetuned")
tokenizer.save_pretrained("./neo_persuasion_finetuned")

```



```

/tmp/ipython-input-3710156766.py:1: FutureWarning: `tokenizer` is deprecated
  trainer = Trainer(
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxError:
  | _ | | ' _ \ / _ ' / _ ' | _ / - _ )
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally:
wandb: You can find your API key in your browser here: https://wandb.ai/
wandb: Paste an API key from your profile and hit enter: .....
wandb: WARNING If you're specifying your api key in code, ensure this code
wandb: WARNING Consider setting the WANDB_API_KEY environment variable,
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: sudiptamitra945 (sudiptamitra945-nshm) to
Tracking run with wandb version 0.22.2
Run data is saved locally in /content/wandb/run-20251015_074142-jwwxbrss
Syncing run genial-cloud-15 to Weights & Biases (docs)
View project at https://wandb.ai/sudiptamitra945-nshm/huggingface
View run at https://wandb.ai/sudiptamitra945-nshm/huggingface/runs/jwwxbrss
 [15498/15498 43:40, Epoch 3/3]

```

Step	Training Loss
500	3.078300
1000	2.956200
1500	2.974800
2000	2.877900
2500	2.896600
3000	2.831700
3500	2.854000
4000	2.778800
4500	2.780700
5000	2.789200
5500	2.575600
6000	2.465300

```

from transformers import pipeline

generator = pipeline("text-generation", model="./neo_persuasion_finetuned")

prompt = "User: I'm not sure if small donations make any difference."
response = generator(prompt, max_length=80, do_sample=True, temperature=0.8)
print(response[0]['generated_text'])

```

```

Device set to use cuda:0
Truncation was not explicitly activated but `max_length` is provided a sp
Both `max_new_tokens` (=256) and `max_length` (=80) seem to have been set.
User: I'm not sure if small donations make any difference. Small donation

```